Java_script-2_<No-1>_Day-5_Hands_on_Gunta_Divya

Problem 1: Greeting Generator (Level-1)

Scenario

A simple webpage should greet users based on the name they enter. This helps beginners understand how JavaScript functions accept parameters and how variables behave inside functions.

📌 Requirements

- Create an input box to enter a user's name.
- Create a button labeled "Generate Greeting".
- When the button is clicked:
- A JavaScript function should accept the name as a parameter.
- Display a greeting message like:
  "Hello, Rahul! Welcome to our website."
- Display the greeting inside a <p> element.

🛠 Technical Constraints

- Use only vanilla JavaScript (no libraries).
- Use function keyword (not arrow functions for this task).
- Use document.getElementById() for DOM manipulation.
- Variable for the greeting message must be declared inside the function.

🎯 Learning Outcome

You should be able to:

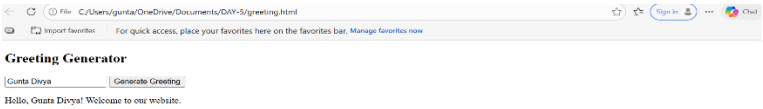
- Understand how functions receive input using parameters.
- Learn local variable scope.
- Manipulate webpage elements using the DOM.

Code:-

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <title>Greeting Generator</title>
</head>
```

```
<body>

    <h2>Greeting Generator</h2>

    <input type="text" id="nameInput" placeholder="Enter your name">

    <button onclick="handleGreeting()">Generate Greeting</button>

    <p id="greetingMessage"></p>

    <script>
        function handleGreeting() {
            var name = document.getElementById("nameInput").value;
            generateGreeting(name);
        }

        function generateGreeting(name) {
            var greeting = "Hello, " + name + "! Welcome to our website.";
            document.getElementById("greetingMessage").innerText = greeting;
        }
    </script>

</body>
</html>
```

Output:-



Code Explaination:-

this Greeting Generator program creates a simple interactive webpage where the user enters their name in an input box and clicks the **Generate Greeting** button. When the button is clicked, the handleGreeting() function is executed, which reads the user's input using document.getElementById() and passes the name as a parameter to another function called generateGreeting(). Inside generateGreeting(), a local variable is declared to store the greeting message that includes the user's name. This message is then displayed dynamically inside a <p> element using DOM manipulation, without refreshing the page. The program helps beginners understand how functions accept

parameters, how local variables work within functions, and how JavaScript interacts with HTML elements to update content dynamically.

Java_script-2_<No-2>_Day-5_Hands_on_Gunta_Divya

Scenario

A webpage needs to display basic user information stored inside a JavaScript object.

📌 **Requirements**

Create a JavaScript object user with properties:

- name
- age
- city

Create a function displayUserInfo(userObj):

- Accepts the object as a parameter.
- Displays user details in separate <p> elements.

A button click should trigger the function.

🛠 **Technical Constraints**

- Object properties must be accessed using dot notation.
- Function should not use global variables.
- DOM elements should be updated dynamically.

🎯 **Learning Outcome**

You will be able to:

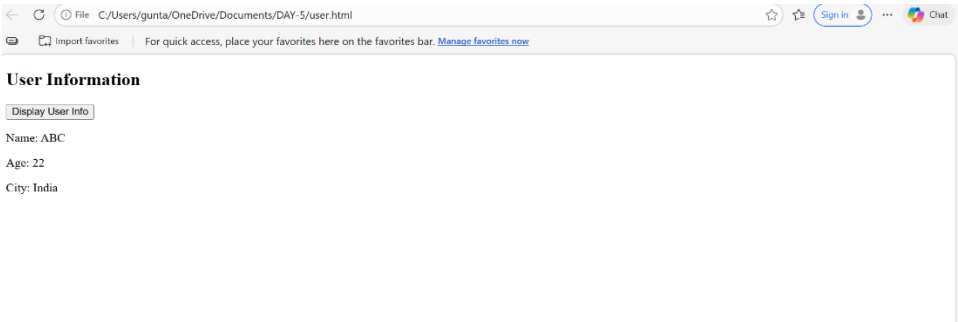
- Understand JavaScript objects.
- Pass objects to functions.
- Display object data dynamically on a webpage.

Code:-

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>User Info Display</title>
</head>
<body>

    <h2>User Information</h2>
```

```
    <button onclick="showUser()">Display User Info</button>

    <div id="output"></div>

    <script>
        const user = {
            name:"ABC",
            age: 22,
            city: "India"
        };
        function displayUserInfo(userObj) {
            const outputDiv = document.getElementById("output");
            outputDiv.innerHTML = "";

            const namePara = document.createElement("p");
            namePara.textContent = "Name: " + userObj.name;

            const agePara = document.createElement("p");
            agePara.textContent = "Age: " + userObj.age;

            const cityPara = document.createElement("p");
            cityPara.textContent = "City: " + userObj.city;

            outputDiv.appendChild(namePara);
            outputDiv.appendChild(agePara);
            outputDiv.appendChild(cityPara);
        }
        function showUser() {
            displayUserInfo(user);
        }
    </script>

</body>
</html>
```

Output:-



Code Explaination:-

This webpage displays basic user information stored inside a JavaScript object. A user object is created with the properties name, age, and city. When the Display User Info button is clicked, the showUser() function is triggered, which passes the user object as a parameter to the displayUserInfo() function. Inside this function, the object properties are accessed using dot notation and displayed dynamically on the webpage by creating separate <p> elements for each detail. The DOM is updated without refreshing the page, helping to demonstrate how objects can be passed to functions and how their data can be shown dynamically using JavaScript.

Java_script-2_<No-3>_Day-5_Hands_on_Gunta_Divya

Build a counter application where users can increment and reset a value. This problem focuses on variable scope and DOM manipulation.

📌 Requirements

Display a counter value starting from 0.
Create two buttons:

- **Increment**
- **Reset**

Use:

- A global variable to store counter value.
- A function incrementCounter(step) that:
  - Accepts step value as a parameter.
  - Updates the counter.

Reset button should reset the counter to 0.

🛠 Technical Constraints

- Counter value must be maintained outside the function (global scope).
- DOM updates must happen inside functions only.
- No inline JavaScript in HTML.

🎯 Learning Outcome

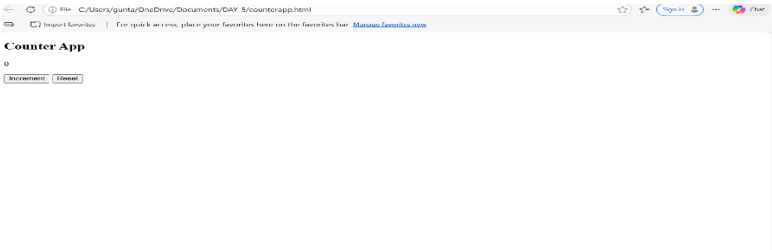
Learners should be able to:

- Understand variable scope (global vs local).
- Apply function parameters in real scenarios.
- Control UI behavior using JavaScript functions.

Code:-

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Counter App</title>
</head>
<body>

    <h2>Counter App</h2>
    <p id="counterValue">0</p>

    <button id="incrementBtn">Increment</button>
    <button id="resetBtn">Reset</button>

    <script>
        let counter = 0;

        function updateDisplay() {
            document.getElementById("counterValue").textContent = counter;
        }

        // Increment function with parameter
        function incrementCounter(step) {
            counter += step;
            updateDisplay();
        }

        function resetCounter() {
            counter = 0;
            updateDisplay();
        }

        document.getElementById("incrementBtn").addEventListener("click", function () {
            incrementCounter(1);
        });

        document.getElementById("resetBtn").addEventListener("click", resetCounter);
    </script>

</body>
</html>
```

Output:-



Code Explaination

This Counter App displays a numeric value on the webpage and allows the user to increment or reset it using buttons. A variable counter is used to store the current count. The updateDisplay() function updates the value shown inside the <p> element using DOM manipulation. The incrementCounter(step) function accepts a parameter and increases the counter by that value, demonstrating how functions can work with parameters. The resetCounter() function sets the counter back to zero. Event listeners are added to the buttons so that clicking Increment increases the counter by 1 and clicking Reset resets it. The page updates dynamically without refreshing, helping to understand functions, parameters, variables, and DOM updates.

Java_script-2_<No-4>_Day-5_Hands_on_Gunta_Divya

Create a mini student profile system where details are stored in an object and displayed dynamically using DOM manipulation.

📌 Requirements

Create a student object with:

- name
- rollNo
- marks

Create a function updateStudentProfile(studentObj):

- Accepts the object as a parameter.
- Displays details in a styled .

Add another function updateMarks(newMarks):

- Updates marks and refreshes the display.

🛠 Technical Constraints

- Student object must be declared in global scope.
- Functions should update object values using parameters.
- DOM should update without page refresh.

🎯 Learning Outcome

You will be able to:

- Deep understanding of object manipulation.
- Function interaction with shared data.
- Real-world use of DOM updates and scope handling.

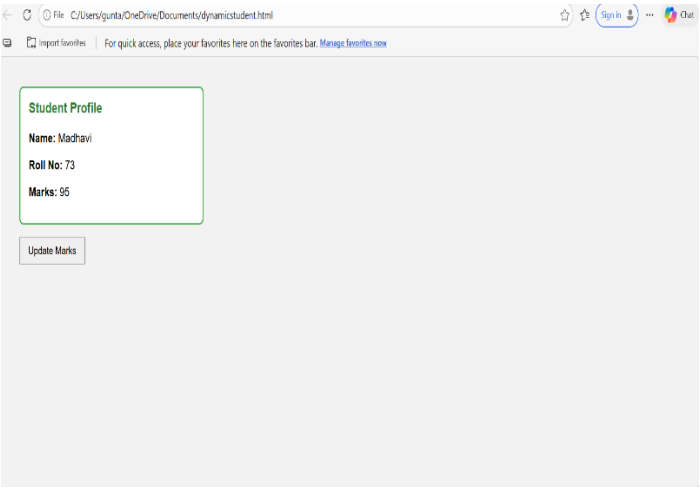
CODE:-

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <title>Mini Student Profile System</title>



  <style>

    body {

      font-family: Arial, sans-serif;

      background-color: #f2f2f2;

      padding: 30px;

    }



    .profile {

      width: 300px;

      padding: 15px;

      border: 2px solid #4CAF50;

      border-radius: 8px;

      background-color: #ffffff;

    }



    .profile h3 {

      margin-top: 0;

      color: #2e7d32;

    }



    button {
```

```
      margin-top: 15px;
      padding: 8px 14px;
      font-size: 14px;
      cursor: pointer;
    }
  </style>
</head>

<body>

  <div id="studentProfile"></div>

  <button onclick="updateMarks(92)">Update Marks</button>

  <script>
    let student = {
      name: "Madhavi",
      rollNo:73,
      marks:95
    };
    function updateStudentProfile(studentObj) {
      const profileDiv = document.getElementById("studentProfile");

      profileDiv.innerHTML = `
        <div class="profile">
          <h3>Student Profile</h3>
          <p><strong>Name:</strong> ${studentObj.name}</p>
          <p><strong>Roll No:</strong> ${studentObj.rollNo}</p>
          <p><strong>Marks:</strong> ${studentObj.marks}</p>
        </div>
      `;
```

```
    }
    function updateMarks(newMarks) {
      student.marks = newMarks;
      updateStudentProfile(student);
    }
    updateStudentProfile(student);
  </script>
</body>
</html>
```

OUTPUT:-



Code Explaination:-

This program creates a simple greeting generator where the user enters their name in an input box and clicks the Generate Greeting button. When the button is clicked, a JavaScript function is executed that reads the entered name using document.getElementById(). Inside this function, a local variable is used to create a greeting message that includes the user's name. The greeting is then displayed dynamically inside a <p> element on the webpage without refreshing the page. This example helps in understanding how functions accept input through parameters, how local variables work, and how the DOM is used to update webpage content.